Lab 1 - Instructions

Basics of numpy and matplotlib

Software 1 - Mathematics

Information and Communication Technology

Metropolia University of Applied Sciences

Understanding what you'll create

In this lab, you will:

- Create an overview graph showing the entire function (Problem 1)
  - This helps you see the "big picture" of what the function looks like
  - You'll use this to locate where intersection points might be
- Create zoomed graphs of each intersection point (Problem 2)
  - Think of this like using a magnifying glass on each area where the function crosses y = 1
  - Each intersection point needs its own zoomed view

Using AI tools

Modern integrated development environments (IDEs), like PyCharm, include AI tools for help, explaining and even documenting the code snippets. Before jumping into the AI companion's world, first try to solve these problems by yourself. It is highly recommended we make some self-thinking effort to understand the problem more thoroughly. However, if you can't persist the temptation, please report which AI tools you used and how.

For more information, see: The use of AI in learning tasks – Traffic Light Model (pdf). Follow the yellow line.

Setup

1. **Create a Jupyter project for your PyCharm environment**
   - In PyCharm, select **File > New Project**
   - Select project type **Jupyter**
   - Change location **..\MathematicsPythonLabs**
   - Check **Create a simple Jupyter Notebook**
   - Select interpreter type **Project venv**
   - Python version **Python 3.13.15** (for example)
   - Click **Create**

Use this project to all your Mathematics Python Labs for the entire study year!

2. **Download Lab 01 My Solutions.ipynb notebook and labx.py module**
   - Go to OMA assignment
   - Download Lab 01 My Solutions.ipynb to your project folder
   - Download `labx.py` file, save it to the SAME FOLDER as the notebook

   If these files are in different folders, you'll get:

   ```
   ModuleNotFoundError: No module named 'labx'
   ```

3. **Open the notebook and fill in your information**

   ```
   name = "Your Full Name"       # Write your name as a string
   student_id = 12345678         # Change to YOUR actual student ID
   date_returned = "DD.MM.YYYY"  # Example: "15.10.2025"
   ```

   - name must be in quotes (it's a string)
   - student_id must be just numbers, NO quotes
   - date_returned format: "DD.MM.YYYY" with quotes

3. **Run the import cell only once**

   ```
   from IPython.display import Markdown, display
   import numpy as np
   import matplotlib.pyplot as plt
   import labx as mx
   ```

   Tip: After running this cell, you don't need to run it again. All imports are now available throughout your notebook.

   If you get an error

   - ModuleNotFoundError: No module named 'labx' → labx.py is not in the same folder as your notebook
   - NameError: name 'np' is not defined → You didn't run the import cell above

Problem 1 - Drawing the function

In the first problem your aim is to draw a given function.

The function is in the `labx` module that you imported. You can call the function with the following code:

```
y = mx.lab_1_problem_1(x, student_id)
```

where

- x is a numerical array containing values for x
- student_id is your student number you defined above
- y is the output of the function, it is a numerical array

Each of you get slightly different output values, so be careful when you collaborate with your classmates!

Step-by-step instructions

1. **Create the x array**

   Write code that creates a numerical array for $x \in [-10,10]$ with step size of 0.001

   Tip: Use `np.arange(-10, 10, 0.001)` - notice the THREE parameters!

   Important: The range must include BOTH negative and positive values: from -10 to +10.

2. **Calculate y-values**

   Use the provided function to calculate y:

   ```
   y = mx.lab_1_problem_1(x, student_id)
   ```

3. **Create the plot**

   Make a graphical plot of the function showing the ENTIRE range.

4. **Decorate your graph**

   Add these elements to make your graph professional:

   - Title (e.g., “Function f(x) for student 12345678”)
   - X-axis label (e.g., “x”)
   - Y-axis label (e.g., “f(x)” or “y”)
   - Grid lines using `plt.grid(True)` - this helps read values from the graph!

What your graph should show

- The complete function from x = -10 to x = 10

- Smooth curve (not jagged - this is why we use step size 0.001)
- Grid lines to help locate where the function crosses y = 0

Example of what NOT to do

- `x = np.arange(-10, 10)` - Missing step size! This creates only 20 points
- `x = np.arange(0, 10, 0.001)` - Missing negative values!
- Manual tick marks instead of grid lines - makes the graph harder to read

Problem 2 - Finding intersection points graphically

What is an intersection point?

An intersection point is a value of x where `f(x) = c`. Graphically, this is where the function crosses the horizontal line where y = c.

Your task

Step 1: Examine your graph from Problem 1

- Look at where the function crosses the line y = 1
- Count how many times it crosses - each crossing is an intersection point
- In this problem your function has exactly two crossing points

Step 2: Create separate zoomed graphs for EACH intersection point

- For each cross point you found, create a NEW zoomed graph that shows the crossing point clearly.

Important notes

- You do NOT need to recreate x and y arrays - they are already in memory
- You DO need to create new plot code for each zoomed view
- Create as many graphs as there are roots (usually 2 graphs)

How to zoom in

```
# Example: Zooming in on a range near x = -7
plt.figure(figsize=(7, 5))
plt.plot(x, y)
plt.xlim(-7.5, -6.5) # Zoom x-axis to show area around x = -7
plt.ylim(0, 2) # Zoom y-axis to show area around y = 1

# Shows the y=1 line
plt.axhline(y=1, color='r', linestyle='--')
plt.xlabel('x')
```

```
plt.ylabel('f(x)')
plt.title('Point 1 - Zoomed view')
plt.grid(True)
plt.show()
```

Step 3: Read the intersection value from your zoomed graph

- The intersection point is the x-value where the function crosses y = 1
- Read the value with two decimal accuracy (e.g., x = -6.70)
- You should be able to clearly see this from your zoomed, gridded graph

Checklist for Problem 2

[ ] I found ALL the intersection points by examining the overview graph

[ ] I created a separate graph for each crossing (2 graphs)

[ ] Each zoomed graph shows the y = 1 line clearly

[ ] Each zoomed graph has grid lines

[ ] I can read each x-value to 2 decimal places from my graphs

Common mistakes to avoid

- Creating only ONE zoomed graph when there are multiple points
- Zooming but not including y = 1 in the visible range
- Not zooming in enough - you should be able to read the crossing point value with enough accuracy
- Trying to create new x and y arrays (they're already in memory!)

Troubleshooting common issues

My graph looks jagged/rough

- You forgot the step size in `np.arange()`
- Use: `x = np.arange(-10, 10, 0.001)` with all three parameters

My graph shows only positive x values

- You wrote `np.arange(0, 10, 0.001)` instead of `np.arange(-10, 10, 0.001)`
- The range must start at -10, not 0

I can't find all the intersection points

- Look at your Problem 1 graph carefully - how many times does it cross y = 1?
- Create a separate zoomed graph for each crossing
- The function has 2 intersection points

My zoomed graph is empty

- Your xlim/ylim range doesn't include where the function is
- Check your Problem 1 graph to see approximately where the intersection points are
- Make sure your ylim includes 0, e.g., `plt.ylim(-1, 1)`

ModuleNotFoundError: No module named 'labx'

- The labx.py file is not in the same folder as your notebook
- Download it from OMA and move it to your project folder

Grid lines make my graph hard to read

- Don't use manual x-ticks or y-ticks
- Just use `plt.grid(True)` - that's all you need!